MATES: Taste-Based Social Discovery Platform

Project Proposal for UCS503P
Submitted to: Anushka Ma'am
Thapar Institute of Engineering and Technology

Tulika Jain, CSED* Mukul, CSED† Pranav Tiwari, CSED‡

August 21, 2026

1 Higher-order goal

This project aims to improve social discovery by helping users find and connect with people who share similar tastes across multiple domains of entertainment and interest, rather than a single category such as music alone. While the broader ambition is to rethink how people find community online, the immediate engineering objective is to translate *taste similarity* into a measurable, explainable compatibility score delivered through a working software system.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core taste-matching workflow (Profile → Taste Signature → Compatibility Score) and validating it with real users within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations, starting with a simple weighted-similarity matching algorithm before attempting any NLP/embedding-based approach.
- Success is defined by what can be delivered and measured in the first iteration: a working "Find My People" flow with an explainable match score, then improved based on user feedback.

3 Problem Statement

Existing social platforms are typically organized around a single content category (e.g., music, photos, short video) or around a follower-based model that optimizes for reach rather than genuine compatibility. This creates several pain points for users who want to find people who actually share their taste:

- **Fragmentation:** a person's taste is spread across separate apps, one for music, one for movies, one for books, with no unified representation of "who they are."

*Roll No: 1024030xxx, Email: tjain_be24thapar.edu
†Roll No: 1024030xxx, Email: mukul_be24thapar.edu
‡Roll No: 1024030xxx, Email: ptiwari_be24thapar.edu

- **Opaque recommendations:** existing "suggested for you" features rarely explain *why* two users were matched, reducing trust in the suggestion.
- **Follower-centric design:** following someone is a one-directional, low-commitment action that does not guarantee any shared interest, leading to noisy, low-relevance feeds.
- **Weak niche discovery:** users with specific or unusual combinations of interests (e.g., dark academia aesthetics, psychological thrillers, and Formula 1) struggle to find others who share that exact combination.

Why this matters now (contextual relevance): As content categories multiply and attention fragments across apps, a unifying, cross-domain notion of "taste" is increasingly useful for meaningful (rather than purely algorithmic) social discovery.

Positioning relative to existing tools: single-category platforms (e.g., Letterboxd for film, Goodreads for books) already let users track taste within one domain, and dating/social apps increasingly surface shared interest tags as a matching signal. What this project adds is not a new content category, but (i) a *cross-domain* taste representation spanning multiple interest types at once, and (ii) an *explainable* compatibility score that shows the specific shared interests behind a match, rather than an opaque "suggested for you" ranking.

4 Proposed Solution

4.1 Overview

Build a **mobile-first** "Taste Discovery" platform with the following components:

- **Taste Signature:** users build an interest profile from keywords spanning music, movies, and books at launch, with additional categories (memes, games, shows, sports, food, photography, custom/niche tags) added in later iterations.
- **Content Drops:** lightweight posts through which users share and tag content they like, categorized by domain.
- **Taste-based matching engine:** computes a per-category and overall compatibility score between users based on shared keywords and content.
- **Find My People:** a ranked, explainable list of recommended users, each annotated with the specific shared interests driving the match.
- **Connection system:** opt-in, bidirectional connections (request/accept) rather than one-directional following, gating private content behind mutual connection.
- **Communities:** niche, tag-based groups (e.g., `#DarkAcademia`, `#F1Girls`) for users to congregate around specific shared interests, deferred to a subsequent iteration.

4.2 Core workflow

1. User signs up, creates a profile, and selects interest keywords to build their Taste Signature.
2. User explores and engages with content across the launch categories (music, movies, books) through likes, saves, and Drops.
3. System computes per-category and overall compatibility scores against other users.
4. User opens "Find My People" and views ranked matches with an explainable "why you matched" breakdown.
5. User sends a connection request; once accepted, the match's full profile and interest details unlock.

4.3 Operational constraints for an educational setting

- Keep the matching algorithm computationally lightweight (weighted keyword/content overlap) for the MVP; avoid dependence on research-grade embedding models initially.
- Design for deployability using common managed backend/database hosting and free-tier external content APIs (TMDB, Spotify, Google Books).
- Keep authentication, storage, and moderation simple but secure by design from the first iteration, since the platform handles personal interest and chat data.

5 Solution Approach

This is an engineering course project, so we focus on a correct, secure, and scalable matching-and-connection workflow rather than novel recommendation-system research.

5.1 Taste-matching

The MVP matching algorithm will be heuristic-based:

- Represent each user's Taste Signature as a set/vector of weighted keywords across categories.
- Compute per-category similarity (e.g., Jaccard or cosine overlap on keyword sets) and combine into a weighted overall compatibility score, without claiming state-of-the-art accuracy.
- Always surface the specific shared keywords/content behind a match, so recommendations are explainable rather than opaque.

5.2 System architecture

The platform follows a layered client-API-database architecture, chosen for its clear separation of concerns and straightforward path to horizontal scaling.

Client layer

- **Flutter mobile client:** a single cross-platform codebase covering profile, explore, matching, connections, chat, and community screens.
- Communicates with the backend exclusively over HTTPS REST endpoints (WebSockets are added later for real-time chat); no direct database access from the client.
- Local state management (Riverpod/Provider) handles UI state, while all persistent state lives on the backend.

Backend API layer

The backend is a FastAPI service, organized into logically separate modules that can later be split into independent services if load requires it:

- **Auth service:** registration, login, token issuance/verification (via Firebase Authentication), and session management.
- **Profile service:** CRUD operations on user profiles and Taste Signature keywords.
- **Content service:** Drops (create/view/like/save) and category-specific content metadata pulled from external APIs.

- **Matching service:** computes per-category and overall compatibility scores, and serves the ranked "Find My People" list with match explanations.
- **Connections service:** connection requests, accept/reject, and block/report actions.
- **Notification service:** generates in-app notifications for connections, likes, and new matches.

Each module exposes its own set of REST routes under a common API gateway (a single FastAPI app for the MVP), with clear internal boundaries so modules can be extracted into separate deployable services later without redesigning the API contract.

Data layer

- **Primary database:** MongoDB (hosted on MongoDB Atlas) storing user profiles, Taste Signature keywords, Drops, connections, communities, and reports as document collections.
- **Indexing:** keyword and category fields are indexed to keep matching queries and search fast as the user base grows.
- **Object storage:** Cloudinary stores profile pictures and Drop images/media, with only URLs/metadata persisted in MongoDB.

External integrations

The backend calls external content APIs server-side (never exposing third-party API keys to the client) to enrich Drops and profiles with real content metadata:

- **TMDB** for movie titles, posters, genres, and ratings.
- **Spotify API** for songs, artists, albums, and genres.
- **Google Books API** for book titles, authors, and covers.

Example request flow

To illustrate how the layers interact, consider a user opening "Find My People":

1. The Flutter client sends an authenticated `GET /matches` request to the backend.
2. The Auth service verifies the user's token before the request reaches the Matching service.
3. The Matching service fetches the requesting user's Taste Signature from MongoDB, along with a candidate pool of other users' signatures.
4. It computes per-category and overall compatibility scores, ranks candidates, and attaches the specific shared keywords behind each score.
5. The ranked, explainable match list is returned as JSON to the client, which renders it as the "Find My People" screen.

Security boundaries

- The client never talks to MongoDB, Cloudinary, or external content APIs directly; every request is mediated by the backend.
- Secrets (database credentials, API keys) live in backend environment variables, never in the mobile client bundle.
- All traffic between client and backend is over HTTPS, with role-based checks (user vs. moderator vs. admin) enforced at the API layer.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run backend/frontend tests on every change.
- Produce repeatable deployment artifacts to a staging environment.
- Enable safe, frequent releases of incremental matching and social features.

6 Evaluation Criterion

We define success using measurable metrics tied to the core matching-and-connection workflow.

6.1 Primary evaluation metric

Match-to-connection rate (MCR): the percentage of "Find My People" recommendations that result in a sent connection request within a defined window.

- Target: MCR $\geq 45\%$.
- Attribution: measured from database timestamps comparing recommendation impressions to connection-request events.

6.2 Secondary evaluation metrics

- **Connection acceptance rate:** percentage of sent requests that are accepted, as a proxy for match quality.
- **Explainability satisfaction:** pilot users rate the "why you matched" explanation on a simple 1–5 clarity/usefulness scale.
- **Taste Signature completion rate:** percentage of new users who complete a minimum viable Taste Signature (e.g., ≥ 5 keywords across ≥ 2 categories) during onboarding.
- **Match explanation density:** average number of shared-interest tags surfaced per top-5 recommended match. This is measurable directly from stored data without waiting on user behavior, and serves as an early proxy for match quality even before enough connection events accumulate.
- **Reliability:** availability of staging/production for login, profile, and matching endpoints (e.g., $\geq 99\%$ uptime in pilot).

6.3 Pilot validation plan

- Recruit 20–30 pilot users with varied interest profiles to seed a meaningful matching pool.
- To mitigate the cold-start problem inherent to a small pilot, supplement real signups with a curated set of synthetic profiles carrying deliberately overlapping taste keywords, ensuring the matching engine has enough density to produce meaningful, demoable matches even before organic adoption grows.
- Compare MCR and acceptance rate before/after refining the weighting of the similarity algorithm within the iteration window.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of users and matching computations by:
   - decoupling the matching engine from the core API for independent scaling,
   - precomputing/caching compatibility scores instead of recomputing on every request,
   - using indexed queries on keyword and category fields.
2. **Operationally deployable (practically):** The system should run on common hosting:
   - managed document database (MongoDB Atlas),
   - containerized or platform-hosted backend deployment,
   - CI/CD pipeline for staging/production releases.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a mobile/web project:

- Mobile framework (Flutter) for cross-platform client development.
- Web framework (FastAPI) for REST APIs.
- Managed document database (MongoDB Atlas).
- Authentication service (Firebase Authentication).
- Object storage for images/memes (Cloudinary).
- Established similarity/matching libraries (NumPy, scikit-learn) for the MVP algorithm.
- External content APIs (TMDB, Spotify, Google Books) to avoid building content catalogs from scratch.
- Test frameworks (Pytest, Flutter test) and CI runners for staging deployment.

We will use these mature components to focus on matching-workflow design, explainability, and security rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- User authentication and profile creation
- Taste Signature keyword selection across music, movies, and books
- Content Drops: create, view, like, and save
- Weighted-similarity compatibility scoring between users
- "Find My People" ranked list with explainable match reasons
- Connection requests: send, accept, reject

- Privacy settings (public/private profile, hide/show interests)
- Block/report functionality
- CI: build, backend unit tests, linting; CD to staging

The MVP deliberately excludes chat, memes, and communities so that the first iteration's demo stays centered on the core loop: building a Taste Signature, computing an explainable match, and connecting.

9.2 Subsequent deliverables

- Basic one-to-one chat between connected users
- Meme category: upload, tag, and share
- Niche communities: create/join, community posts and discussions
- Admin/moderator dashboard for reports and content moderation
- Notification layer for connections, messages, likes, and new matches
- Semantic/embedding-based matching (Sentence Transformers, FAISS) as a research-grade upgrade to the MVP keyword-overlap algorithm
- Vibe Rooms (temporary interest-based real-time spaces) and group chats

10 Risks and mitigations

- **Data privacy / consent.** Collecting personal interest, profile, and chat data creates exposure if mishandled. We minimize what is collected and keep sensitive data out of the client: attachments stay optional, passwords are never stored in plaintext, and secrets live in backend environment variables.
- **Cold-start matching.** With few pilot users, compatibility scores may be sparse or low-quality. We ensure enough match density by seeding a diverse pilot cohort alongside synthetic profiles with overlapping taste keywords, and lowering the minimum-keyword threshold for early matches.
- **Content moderation risk.** User-uploaded images and Drops could surface inappropriate content, a risk that grows once memes and chat are added later. We build moderation in from the first iteration via upload validation, user reporting flows, and admin review tools.
- **Evaluation measurement ambiguity.** Without clear definitions, metrics like match-to-connection rate become hard to attribute to real behavior. We solve this by instrumenting recommendation-impression and connection-request events with precise timestamps before development begins.
- **Operational issues in pilot.** Deployment or infrastructure issues during the pilot could cause downtime and hurt the reliability metric. We catch these early by using a staging environment continuously and ensuring CI/CD produces repeatable, tested deployments.

11 Summary

This project proposes a deployable taste-based social discovery platform that helps users find and connect with people who share similar interests across multiple content domains, rather than a single category or a follower-based model. It meets the purpose by emphasizing time-to-value through rapid iterations starting with a lightweight, explainable matching algorithm, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially match-to-connection rate). It remains focused on engineering workflow, security, and scalability readiness in its MVP, while reserving semantic/embedding-based matching as a well-scoped future enhancement rather than a first-iteration research dependency.